Instituto Federal de Educação, Ciência e Tecnologia do Rio Grande do Sul — Campus Rio Grande
Tecnologia em Análise e Desenvolvimento de Sistemas
Princípios e Padrões de Arquitetura

PROJETO INTEGRADOR · ENTREGA 2

Proposta Arquitetural

Community Bank: plataforma de um banco comunitário
abordagem, componentes, decisões, alternativas, trade-offs
e a funcionalidade “Depósito em conta corrente” implementada em camadas

Francine Gonçalves Nunes Moraes
Professora: Beatriz Costa

Branch da entrega:
`github.com/FrancineGnMoraes/community-bank-api/tree/entrega-2-proposta-arquitetural`

Rio Grande, 9 de outubro de 2026

1 Resumo da proposta

EM UMA FRASE

O Community Bank será um **monólito modular** no estilo **cliente-servidor**: clientes web leves conversam por HTTP/JSON com uma única API em Node.js + TypeScript + Express. Por dentro, cada módulo de negócio é organizado em **quatro camadas** (apresentação, aplicação, domínio e infraestrutura), e as dependências apontam para o domínio (**Inversão de Dependência**).

A proposta responde aos drivers levantados na Entrega 1 (Visão). Ela foi construída com o **ADD (Attribute-Driven Design)**, em uma primeira rodada com duas iterações:

Iteração	Meta	Drivers tratados	Elemento refinado
1	Estabelecer a estrutura geral do sistema	R1, R3, R4, R6, OB5, CRN-1, QA-06, QA-10	O sistema inteiro
2	Garantir dinheiro correto e rastreável na primeira operação financeira	QA-04, QA-05 (parcial), CRN-2, CRN-3, CRN-4	Módulo Contas, caso de uso Depósito

Os identificadores (QA-xx, R-x, CRN-x, OBx) são os da Entrega 1. QA = cenário de atributo de qualidade; R = restrição; CRN = preocupação arquitetural; OB = objetivo de negócio.

LINK DA BRANCH DESTA ENTREGA

https://github.com/FrancineGnMoraes/community-bank-api/tree/entrega-2-proposta-arquitetural

A branch da Entrega 1 é https://github.com/FrancineGnMoraes/community-bank-api/tree/entrega-1-visao.

1.1 Drivers relembrados

- **QA-02** (H, H) Acesso sem autenticação é negado e registrado.
- **QA-05** (H, H) Falha no meio de uma operação não deixa saldo inconsistente.
- **QA-01** (H, M) Saldo e extrato em até 2 s (p95) no pico.
- **QA-04** (H, M) Toda movimentação é rastreável e imutável.
- **QA-07** (H, M) Prestação de contas confere com os lançamentos.
- **QA-06** (M, M) Nova regra de crédito em até 1 dia, sem alterar controllers.
- **QA-10** (M, B) Regras testadas sem banco e sem HTTP.
- **R1** Node.js + TypeScript + Express · **R4** custo zero · **R6** celular com conexão ruim.
- **OB5** Equipe pequena: o sistema precisa ser simples de operar.

2 Abordagem arquitetural adotada

A proposta combina estilos, padrões e princípios. Cada um resolve uma parte diferente do problema:

Estilo / padrão	Para que serve aqui	Driver atendido
Cliente-servidor com cliente magro	Regras e dados ficam no servidor; o celular só exibe e envia	R6, QA-09, segurança centralizada (QA-02)
Monólito modular	Um único processo e um único deploy, separado por módulos de negócio	R4, OB5, R3 (prazo), *buildability*
Camadas (4 camadas lógicas, 2 tiers físicos)	Separar o que muda por razões diferentes: HTTP, fluxo, regra e tecnologia	QA-06, QA-10, CRN-1

Estilo / padrão	Para que serve aqui	Driver atendido
Inversão de Dependência (Repository como porta)	O domínio define a interface; a infraestrutura implementa	QA-06, QA-10, troca de banco sem afetar regras
MVC no back-end: Controller + Presenter + DTO	Controller adapta HTTP; Presenter formata para exibição (Humble Object)	QA-09, QA-10, Aula 6
Page Controller + middleware central	Um controller por recurso; erros e JSON tratados em um ponto central	CRN-3 (integridade conceitual)
Strategy para políticas de negócio	Limites e regras de crédito como implementações trocáveis	QA-06
Lançamentos imutáveis (ideia de Event Sourcing)	O saldo é a soma dos lançamentos; nada é apagado	QA-04, QA-07

3 Diagrama arquitetural

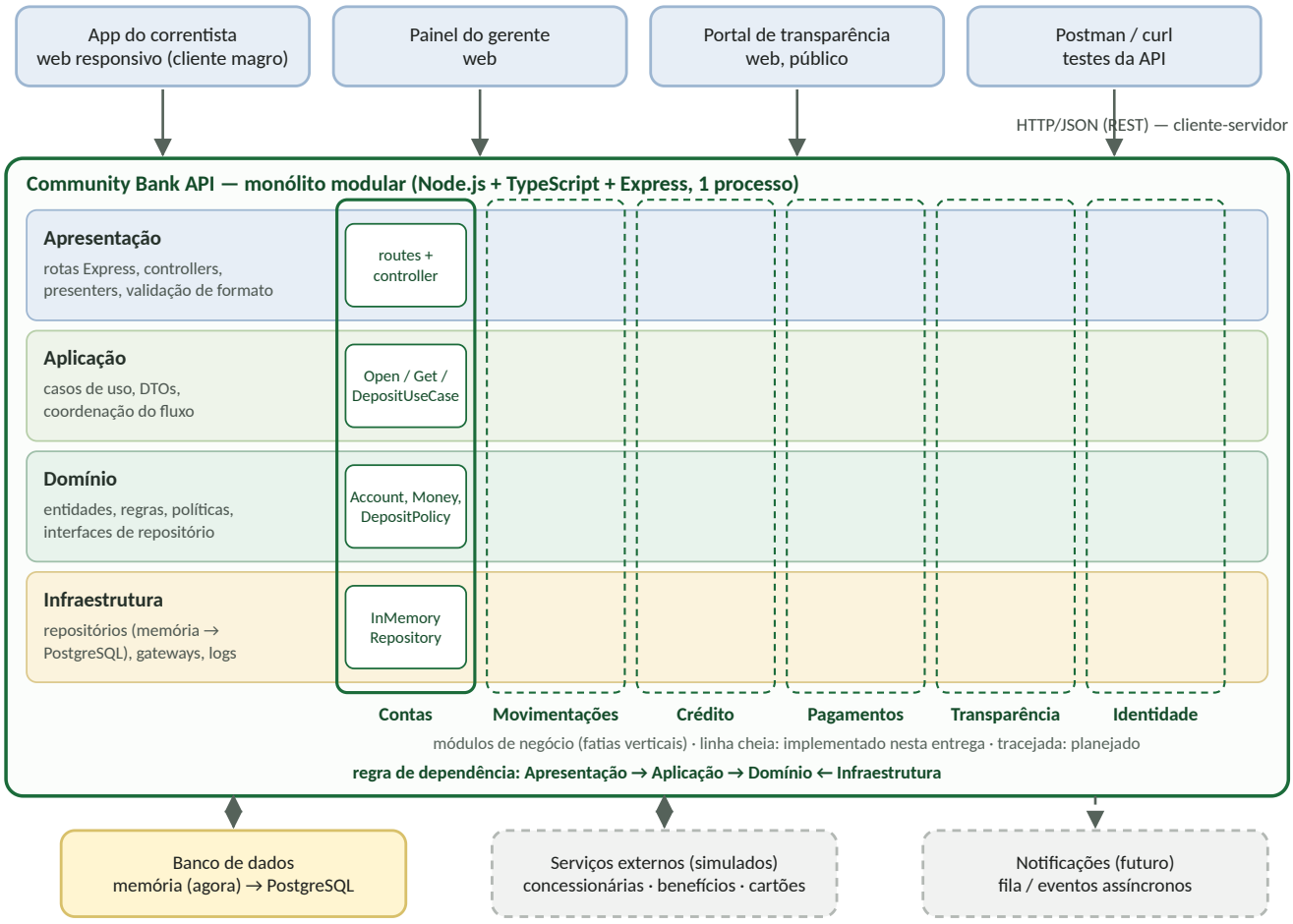


Figura 1 — Visão geral: cliente-servidor por fora, monólito modular em camadas por dentro. Contas é o primeiro módulo implementado.

O diagrama mostra três visões ao mesmo tempo, como pede a Aula 5: **cliente-servidor** organiza a interação (topo), **camadas** organizam as responsabilidades (faixas horizontais) e os **módulos** organizam o negócio (colunas). Cada caso de uso atravessa as camadas de cima para baixo dentro do seu módulo. Assim evitamos o antipadrão *Lasagna*: o código é agrupado primeiro por módulo de negócio (`src/modules/accounts`) e só depois por camada.

4 Módulos e responsabilidades

Módulo	Responsabilidade	Serviços	Situação
Contas	Abertura, consulta, status da conta, depósito e histórico de lançamentos	S1, S2	**Implementado** (abertura, consulta, depósito)
Movimentações	Saque avulso e com cartão, transferências e estornos, sempre com transação	S3	Entrega 3–4
Crédito	Pedido, análise e aprovação de crédito solidário e de consumo sem juros; políticas definidas pelo conselho	S4, S5, S10	Entrega 3–4
Pagamentos	Pagamento de contas e recebimento de benefícios; isola os gateways externos (Adapter)	S8, S9	Entrega 4 (simulado)
Transparência	Prestação de contas diária e semanal, só com totais e sem dados pessoais	S6	Entrega 4
Identidade	Autenticação, perfis (correntista, gerente, conselho) e autorização; trilha de acessos negados	S7	Entrega 3
Shared (núcleo comum)	`Money`, `DomainError`, relógio e gerador de IDs, tratamento de erros HTTP	—	Implementado

4.1 Responsabilidade de cada camada

Camada	Pode	Não pode	No código
Apresentação	Ler parâmetros, validar formato, chamar caso de uso, escolher status HTTP, formatar a saída	Regra de negócio; acessar repositório direto (camada relaxada perigosa, Aula 5)	`presentation/`
Aplicação	Coordenar o caso de uso, buscar e salvar entidades, aplicar políticas, devolver DTOs	Conhecer Express, SQL ou formato de tela	`application/`
Domínio	Entidades, invariantes, objetos de valor, políticas e interfaces de repositório	Depender de qualquer outra camada ou framework	`domain/`
Infraestrutura	Implementar repositórios e gateways, mapear entidade ↔ linha do banco	Decidir regras de negócio	`infrastructure/`

5 Dependências e formas de comunicação

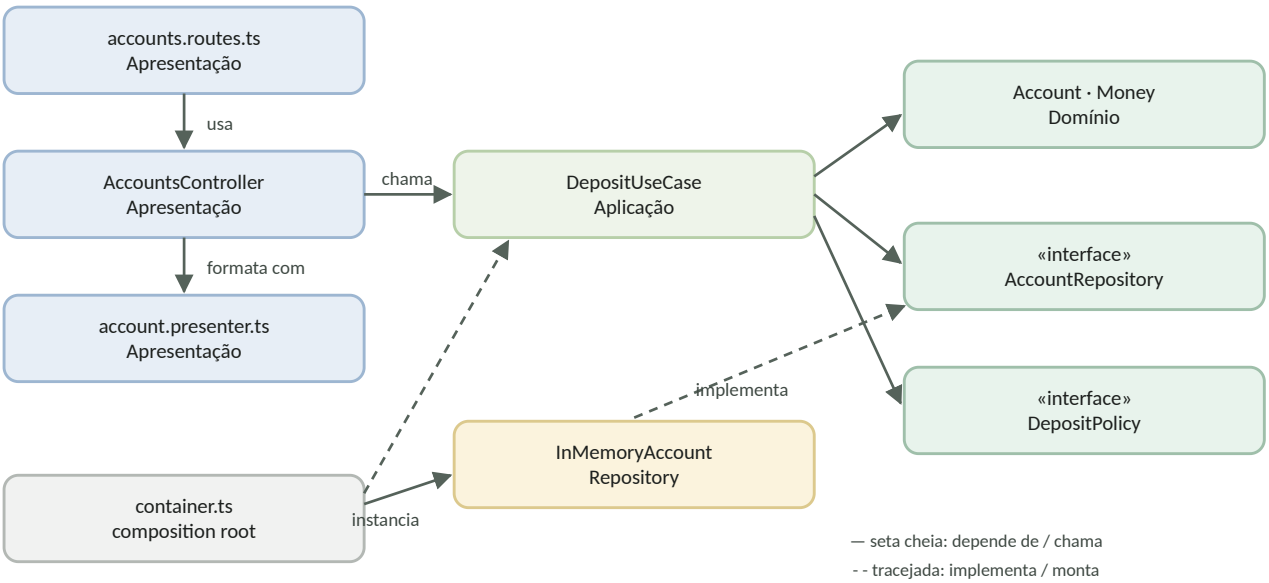


Figura 2 — Dependências de código no módulo Contas. O domínio não importa nada de fora dele; a infraestrutura implementa uma interface do domínio.

De → para	Conector	Modo	Observação
Cliente → API	HTTP/JSON (REST)	Síncrono	Respostas pequenas e com valores já formatados (QA-09)
Controller → Caso de uso	Chamada de método	Síncrono	Entrada já validada no formato; saída é um DTO
Caso de uso → Domínio	Chamada de método	Síncrono	Regras ficam na entidade e nas políticas
Caso de uso → Repositório	Interface `AccountRepository`	Assíncrono (`Promise`)	Já assíncrono para o PostgreSQL entrar sem mudar a assinatura
Módulo → módulo	Interface pública do módulo	Síncrono	Nenhum módulo lê o repositório de outro
API → serviços externos	Adapter / Gateway (HTTP)	Síncrono, com timeout	Isola contratos que não controlamos (R7)
Movimentações → Transparência	Eventos internos (Observer / pub-sub)	Assíncrono	Planejado: a prestação de contas não pode atrasar o depósito

A **composition root** (`src/main/container.ts`) é o único lugar que conhece as classes concretas. Trocar a persistência em memória pelo PostgreSQL significa mudar uma linha nesse arquivo.

6 Relação das decisões com ASRs e atributos de qualidade

ASR	Decisão / tática	Como foi aplicada (ou será)
QA-06 Modificabilidade	Localizar mudanças (coerência semântica), ocultar informação, adiar vínculo (polimorfismo)	`DepositPolicy` como Strategy: uma regra nova é uma nova classe, sem tocar no controller. Testado em `deposit.use-case.test.ts`.

ASR	Decisão / tática	Como foi aplicada (ou será)
QA-10 Testabilidade	Separar interface da implementação, dependências injetáveis, Humble Object	Relógio, IDs e repositório são injetados; 13 testes automatizados rodam em menos de 1 s, sem banco.
QA-04 Auditabilidade	Trilha de auditoria; registros imutáveis	Saldo calculado pelos lançamentos; cada lançamento é congelado (`Object.freeze`) e só existe operação de inclusão.
QA-05 Consistência	Transações (prevenir falhas), validar antes de alterar	Hoje: todas as regras são checadas antes de gravar, e o depósito é salvo de uma vez. Com o PostgreSQL: transação + bloqueio da linha da conta (CRN-8).
QA-02 / QA-03 Segurança	Autenticar e autorizar usuários; limitar exposição; trilha de auditoria	Planejado para a Entrega 3: middleware de autenticação no ponto central e autorização no caso de uso (dono da conta ou gerente da carteira).
QA-01 Desempenho	Reduzir overhead; cache e índices na consulta de saldo	Um único processo, sem chamadas de rede internas. Se o cálculo do saldo pesar, usar snapshot de saldo (Aula 1).
QA-08 Disponibilidade	Detectar falhas (ping/echo), monitor de processo, reinicialização	`GET /health` já existe; um monitor de processo (PM2 ou container com *restart*) reinicia a API.
QA-09 Usabilidade	Separar UI do domínio; feedback ao usuário	Presenter devolve `balanceText: "R$ 150,75"` e mensagens em português simples.
CRN-2 Dinheiro	Objeto de valor imutável	`Money` guarda centavos inteiros; recusa mais de duas casas decimais.
CRN-3 Erros	Integridade conceitual	Um `errorHandler` para toda a API: sempre `{ error: { code, message } }`.

7 Alternativas consideradas

Seguindo a “tabela de alternativas” da Aula 3, comparamos as opções plausíveis antes de escolher.

7.1 Estilo geral

Alternativa	Prós	Contras	Decisão
Microsserviços (um serviço por módulo)	Deploy e escala independentes; isolamento de falhas	Rede entre serviços, consistência distribuída (saga), vários deploys e bancos; custo e operação altos para uma pessoa	**Rejeitada** Nenhum driver pede implantação independente; conflita com R4 e OB5. Seria escolha por *marketability*.
MVC “tradicional” em 3 camadas sem inversão (controller → service → repository concreto)	Mais simples; menos arquivos; familiar	Regras dependem do banco; testar exige banco; tende a *service* gigante	**Rejeitada** Não atende bem QA-06 e QA-10, que são drivers.
Hexagonal completa (portas e adaptadores para tudo)	Domínio totalmente isolado	Mais indireção e interfaces do que o tamanho do projeto pede	**Parcial** Usamos a ideia só onde há variação real (persistência e gateways externos).

Alternativa	Prós	Contras	Decisão
Event-driven como estilo principal	Desacoplamento; bom para a prestação de contas	Fluxo difícil de seguir; consistência eventual em operações de saldo	**Parcial** Só para efeitos secundários (transparência, notificações).
Monólito modular em camadas com DIP	Um deploy; limites claros; testável; evolui para serviços se precisar	Disciplina para não quebrar os limites entre módulos	**Escolhida**

7.2 Persistência

Alternativa	Prós	Contras	Uso
Memória	Rápida; ótima para aula e testes	Perde dados ao reiniciar	**Agora** (protótipo)
Arquivo JSON	Simples; persiste localmente	Concorrência e integridade fracas	Descartada para dinheiro
PostgreSQL	Transações ACID, SQL, maturidade, gratuito	Setup e migrations	**Próximas entregas**
MongoDB	Flexível, documentos	Modelagem de consistência exige cuidado	Descartada: o domínio é relacional e transacional

Como o caso de uso depende só da interface `AccountRepository`, começar em memória não é uma aposta arriscada: é um limite “preparado, mas ainda não implementado por completo” (Aula 5, camadas e limites).

8 Decisões registradas

O diagrama mostra o resultado; a decisão explica por que ele existe (Aula 3, passo 6). Registro curto no formato ADR:

ADR	Decisão	Motivo (rationale)	Consequência
01	Monólito modular cliente-servidor	Equipe de uma pessoa, custo zero, prazo curto; nenhum driver pede distribuição	Um deploy; limites entre módulos dependem de disciplina
02	4 camadas por módulo com Inversão de Dependência	QA-06 e QA-10; impedir regra de negócio no controller	Mais arquivos e indireção (custo aceito)
03	Organizar pastas por módulo e depois por camada	Evitar a *Lasagna Architecture*; manter cada conceito do negócio junto	`src/modules/<módulo>/<camada>`
04	Dinheiro em centavos inteiros (`Money`)	Ponto flutuante erra em centavos; cada centavo importa	Conversão na borda (reais ↔ centavos)
05	Saldo derivado de lançamentos imutáveis	QA-04 e QA-07: todo valor tem origem; estorno é novo lançamento	Calcular o saldo custa mais; snapshot se precisar
06	Persistência em memória atrás de uma interface	Entregar cedo e testar sem banco; PostgreSQL depois	Dados somem ao reiniciar (só nesta fase)
07	Erros de domínio sem HTTP; tradução central	Integridade conceitual; o domínio não conhece Express	Tabela código → status em um único arquivo
08	Relógio e gerador de IDs injetáveis	Testes previsíveis (QA-10)	Parâmetros a mais nos construtores

9 Principais trade-offs

GANHAMOS

- Regras de negócio testáveis sem servidor nem banco.
- Troca de tecnologia (banco, gateway) sem mexer no domínio.
- Um único deploy, simples de operar e barato.
- Histórico completo das movimentações (auditoria).
- Caminho aberto para extrair um módulo como serviço, se um driver pedir.

PAGAMOS

- Mais indireção: um depósito passa por 5 elementos.
- Mais arquivos e algum *boilerplate* (mappers, DTOs).
- Um processo só: se ele cair, tudo cai (mitigado pelo QA-08).
- Escala vertical primeiro; horizontal exige atenção ao estado.
- Saldo calculado pela soma dos lançamentos fica mais caro com o tempo.

Arquitetura é gestão de trade-offs (Aula 1): mais isolamento aumenta a complexidade, mas aqui os drivers de modificabilidade, testabilidade e auditabilidade justificam esse custo, enquanto os de distribuição não justificariam microsserviços.

10 Parte prática: “Depósito em conta corrente”

A funcionalidade nova escolhida foi o **depósito** (serviço S2, caso de uso UC03). É a primeira operação que movimenta dinheiro, então exercita os drivers mais importantes: dinheiro correto, regra no domínio, lançamento rastreável e erro padronizado.

10.1 Regras de negócio

- Só é possível depositar em conta **ativa** (conta bloqueada recusa com `ACCOUNT_NOT_ACTIVE`).
- O valor precisa ser **maior que zero** e ter **no máximo duas casas decimais**.
- Há um **limite por operação** (R$ 5.000,00), definido como política trocável. Acima disso, o correntista é orientado a procurar o gerente.
- Todo depósito gera um **lançamento imutável** com id, valor, descrição e data; o saldo é a soma dos lançamentos.

10.2 Fluxo: Cliente → Controller → Use Case → Domain → Repository → Dados

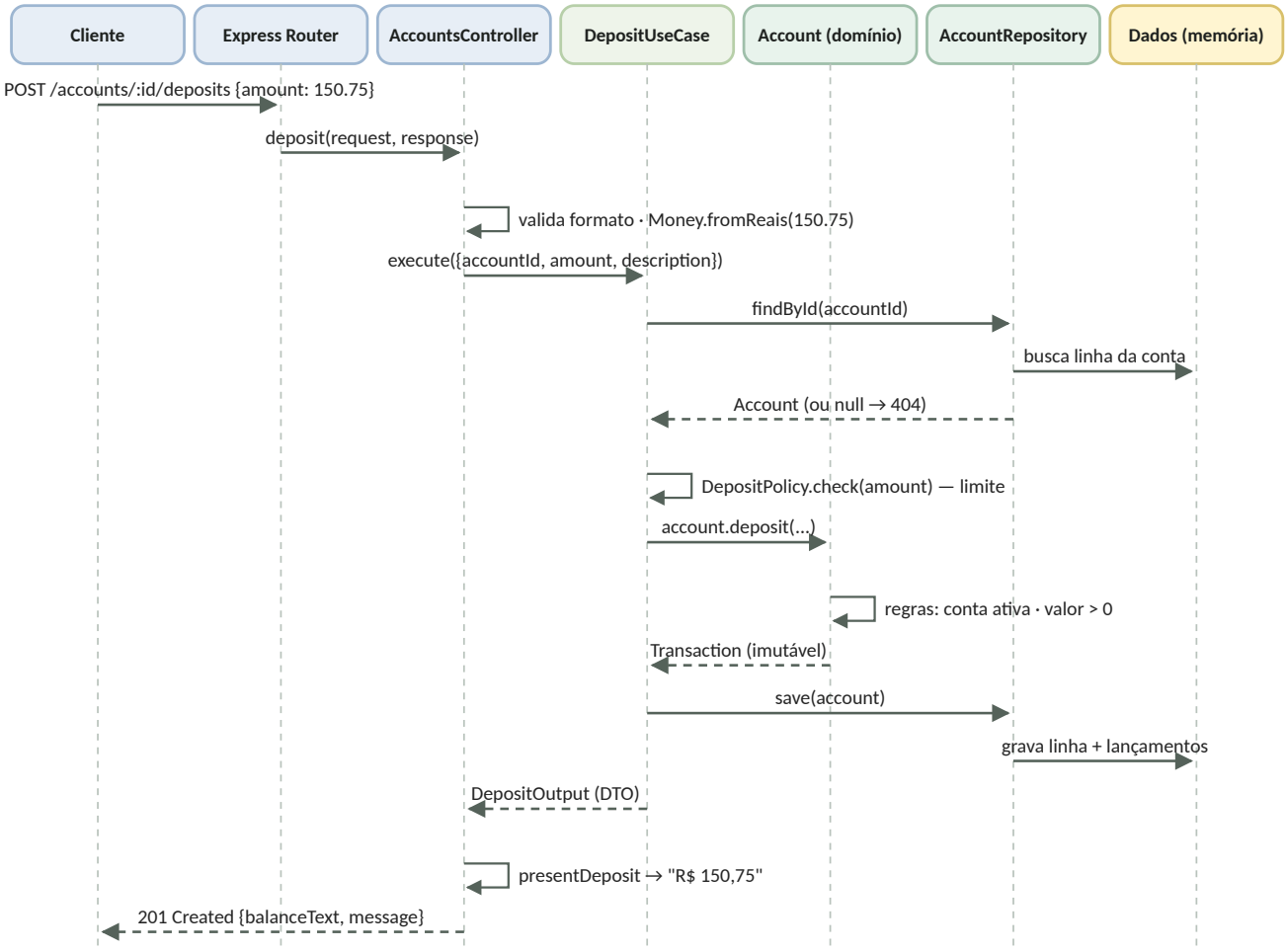


Figura 3 — Sequência de um depósito. Setas tracejadas são retornos.

Etapa	Arquivo	O que faz
Cliente	navegador, app, Postman ou curl	Envia `POST /accounts/:id/deposits` com `{"amount": 150.75}`
Rota	`accounts.routes.ts`	Liga a URL ao método do controller (Page Controller)

Etapa	Arquivo	O que faz
Controller	`accounts.controller.ts`	Confere se `amount` é número, converte para `Money`, chama o caso de uso e devolve 201
Use Case	`deposit.use-case.ts`	Busca a conta, aplica a `DepositPolicy`, pede o depósito à entidade, salva e monta o DTO
Domain	`account.ts`, `money.ts`, `deposit-policy.ts`	Verifica conta ativa e valor positivo; cria o lançamento imutável; calcula o saldo
Repository	`in-memory-account-repository.ts`	Converte a entidade em "linha" (só tipos primitivos) e guarda; faz o caminho inverso na leitura
Apresentação da saída	`account.presenter.ts`	Formata centavos em `"R$ 150,75"` e escreve a mensagem para o usuário

10.3 Trechos principais

Controller: só adapta HTTP. Não há regra de negócio nem acesso ao repositório.

```
deposit = async (request: Request<{ id: string }>, response: Response) => {
  const { amount, description } = request.body ?? {};
  if (typeof amount !== "number") {
    throw new ValidationError("Informe amount como número, em reais. Exemplo: 150.75");
  }
  const result = await this.depositUseCase.execute({
    accountId: request.params.id, amount: Money.fromReais(amount), description
  });
  return response.status(201).json(presentDeposit(result));
};
```

Caso de uso: coordena o fluxo e depende apenas de interfaces.

```
async execute(input: DepositInput): Promise<DepositOutput> {
  const account = await this.accounts.findById(input.accountId);
  if (!account) throw new DomainError("ACCOUNT_NOT_FOUND", "Conta não encontrada.");

  this.policy.check(input.amount);                // Strategy: limite por operação
  const transaction = account.deposit({ ... });   // regra fica na entidade
  await this.accounts.save(account);
  return { transactionId: transaction.id, balanceInCents: account.balance.cents, ... };
}
```

Domínio: a regra mora aqui, sem nenhuma dependência de Express ou de banco.

```
deposit(input: { transactionId: string; amount: Money; description: string; now: Date }):
Transaction {
  if (this.props.status !== "ACTIVE") throw new DomainError("ACCOUNT_NOT_ACTIVE", ...);
  if (!input.amount.isPositive()) throw new DomainError("INVALID_AMOUNT", ...);
  const transaction = Object.freeze({ id: input.transactionId, type: "DEPOSIT", ... });
  this.transactions.push(transaction);           // só inclusão: nada é editado ou apagado
  return transaction;
}

get balance(): Money {                              // saldo = soma dos lançamentos
  return this.transactions.reduce((total, t) => total.add(t.amount), Money.zero());
}
```

10.4 Estrutura de pastas da branch

```
src/
├── app.ts · server.ts                      # como no guia da disciplina
├── main/        container.ts · create-app.ts # composition root
├── shared/
│   ├── domain/  money.ts · domain-error.ts · ports.ts
│   └── http/    error-handler.ts · http-error.ts
└── modules/accounts/
    ├── presentation/   accounts.routes.ts · accounts.controller.ts · account.presenter.ts
    ├── application/    deposit.use-case.ts · open-account / get-account / list-accounts · DTOs
    ├── domain/         account.ts · transaction.ts · account-repository.ts · deposit-policy.ts
    └── infrastructure/ in-memory-account-repository.ts
test/   account.test.ts · deposit.use-case.test.ts · accounts.http.test.ts
```

10.5 Rotas e exemplo de uso

Método	Rota	Descrição	Respostas
GET	`/`, `/health`	Esqueleto da Entrega 1	200
GET	`/accounts`	Lista as contas	200
POST	`/accounts`	Abre conta (`ownerName`, `document`)	201 · 400 · 409
GET	`/accounts/:id`	Consulta conta e saldo	200 · 404
POST	`/accounts/:id/deposits`	**Depósito** (`amount`, `description` opcional)	201 · 400 · 404 · 422

```
curl -X POST localhost:3000/accounts/<id>/deposits -H "content-type: application/json" \
     -d '{"amount": 150.75, "description": "Venda na feira"}'

# 201 Created
{ "transactionId": "1d08d47b-...", "accountId": "6052dbd2-...",
  "amount": 150.75, "amountText": "R$ 150,75", "balance": 150.75, "balanceText": "R$ 150,75",
  "message": "Depósito de R$ 150,75 realizado com sucesso." }

# 422 — acima do limite por operação
{ "error": { "code": "DEPOSIT_LIMIT_EXCEEDED",
             "message": "O depósito passa do limite por operação. Procure o gerente da sua
conta." } }
```

10.6 Testes

Foram escritos 13 testes automatizados com o executor de testes nativo do Node (`node:test`), rodando via tsx, então não foi preciso instalar nenhuma dependência nova. Eles cobrem três níveis:

- **Domínio** (`account.test.ts`): abertura, depósito, valor zero ou negativo, conta bloqueada, histórico protegido e casas decimais.
- **Caso de uso** (`deposit.use-case.test.ts`): persistência do saldo, política de limite e conta inexistente.
- **Ponta a ponta** (`accounts.http.test.ts`): abre conta, deposita e consulta pela API real; confere o formato dos erros e o CPF duplicado.

```
npm install
npm test          # ℹ tests 13 · ℹ pass 13 · ℹ fail 0
npm run typecheck && npm run build && npm start
```

10.7 Relação com a proposta arquitetural

O depósito é a primeira **fatia vertical** do módulo Contas. Foi escolhido porque obriga a pôr em prática, em pequena escala, cada decisão da proposta. Por isso ele também serve para validar a proposta:

Decisão da proposta	Como aparece no depósito
Cliente-servidor com cliente magro (seção 2)	O cliente só envia `{"amount": 150.75}`; validação, regras e formatação ficam no servidor
Camadas com regra de dependência (ADR-02)	Um arquivo por camada; o controller nunca chama o repositório; o domínio não importa Express
Organização por módulo (ADR-03)	Tudo do depósito está em `src/modules/accounts/`
Inversão de Dependência (ADR-02, ADR-06)	`DepositUseCase` recebe `AccountRepository` (interface); a versão em memória é montada no `container.ts`
Modificabilidade (QA-06)	O limite de R$ 5.000,00 é uma `DepositPolicy`; trocar a regra não altera o controller
Dinheiro correto (CRN-2, ADR-04)	`Money.fromReais(150.75)` vira 15075 centavos já na entrada
Auditabilidade (QA-04, ADR-05)	O depósito cria um lançamento congelado; o saldo é recalculado pela soma
Erros padronizados (CRN-3, ADR-07)	Conta inexistente, limite e JSON inválido saem todos como `{ error: { code, message } }`
Testabilidade (QA-10, ADR-08)	Relógio, IDs e repositório injetados permitem testar o caso de uso sem servidor

As próximas funcionalidades (saque, crédito, pagamentos) seguem o mesmo molde. Quando o PostgreSQL entrar, mudam só a camada de infraestrutura e a composition root.

11 Questionário de táticas

Análise do design atual com o questionário baseado em táticas (Aula 3): está suportada? como? qual o risco? por quê?

Tática	Suportada?	Como / onde	Risco	Por quê
Restringir dependências	Sim	Domínio define `AccountRepository`	Baixo	Permite trocar a persistência sem alterar regras
Adiar vínculo (polimorfismo)	Sim	`DepositPolicy` injetada no caso de uso	Baixo	Regra nova sem alterar controllers (QA-06)
Trilha de auditoria	Parcial	Lançamentos imutáveis; falta registrar o autor	Médio	O autor depende da autenticação (Entrega 3)
Transações	Parcial	Validação completa antes de gravar; uma gravação por operação	Alto	Sem banco real ainda; operações com duas contas exigem transação ACID
Autenticar / autorizar	Não	Previsto como middleware + regra no caso de uso	Alto	QA-02 é (H, H); vai para o topo do backlog
Detectar falhas (ping/echo)	Sim	`GET /health`	Baixo	Base para o monitor reiniciar a API (QA-08)
Exceções	Sim	`DomainError` + `errorHandler` central	Baixo	Falhas sinalizadas sempre do mesmo jeito

12 Riscos e próximos passos

Risco / pendência	Ação planejada	Quando
Sem autenticação (QA-02, H/H)	Módulo Identidade: login, perfis, middleware de autenticação, log de acessos negados	Entrega 3
Concorrência no saldo (CRN-8) e consistência (QA-05)	PostgreSQL com transação e `SELECT ... FOR UPDATE` na conta	Entregas 3–4
Autor dos lançamentos ainda não é registrado	Guardar o usuário autenticado em cada lançamento	Entrega 3
Saldo calculado pela soma pode ficar lento	Medir com carga; se precisar, snapshot periódico do saldo	Entrega 4
Limites entre módulos dependem de disciplina	Regra de lint para impedir import entre camadas/módulos proibidos	Entrega 3

BACKLOG ARQUITETURAL ATUALIZADO

Completamente tratados: R1, R2, CRN-1, CRN-2, CRN-3, CRN-4, QA-06 (no depósito), QA-10. **Parcialmente tratados:** QA-04, QA-05, QA-08, QA-09, CRN-6. **Não tratados:** QA-02, QA-03, QA-07, CRN-5, CRN-8.

13 Referências

- BASS, L.; CLEMENTS, P.; KAZMAN, R. *Software Architecture in Practice*. 4. ed. Addison-Wesley, 2021.
- CERVANTES, H.; KAZMAN, R. *Designing Software Architectures: A Practical Approach*. Addison-Wesley, 2016.
- FOWLER, M. *Patterns of Enterprise Application Architecture*. Addison-Wesley, 2002.
- GAMMA, E. et al. *Padrões de Projeto: soluções reutilizáveis de software orientado a objetos*. Bookman, 2000.
- GARLAN, D.; SHAW, M. *An Introduction to Software Architecture*. CMU-CS-94-166, 1994.
- MARTIN, R. C. *Arquitetura limpa: o guia do artesão para estrutura e design de software*. Rio de Janeiro: Alta Books, 2019.
- Material de aula da disciplina Princípios e Padrões de Arquitetura (Aulas 1, 3, 4, 5 e 6) e guia “Community Bank API — Node.js, TypeScript e Express”.